

Nomad

A Functional Language with Borrow Checking

Walid Menou

Abstract

This project implements an interpreter for a functional programming language extended with mutability and borrow checking, investigating affine type systems for resource management. Borrow checking, famously used in Rust, provides strong compile-time guarantees about how values are used and mutated without relying on a garbage collector. A second objective is to explore expressiveness in language design by implementing features such as list comprehensions, a pipe operator, algebraic data types, pattern matching, and Hindley-Milner type inference. The developer should not have to manually annotate types or write redundant code, and should be able to express ideas concisely while still benefiting from strong compile-time guarantees. This report defines the language as it stands. It gives the grammar and documents every construct the interpreter supports, with the type it has and what it does, and it records the work still to come.

Contents

1	Introduction	3
1.1	A first program	3
1.2	Running a program	3
2	Syntax	4
2.1	Grammar	4
2.2	Operators	5
2.3	Layout, comments and lexical rules	6
3	The language	6
3.1	Literals	6
3.2	Variables and bindings	7
3.3	Arithmetic	7
3.4	Comparison and equality	7
3.5	Chained comparison	8

3.6	Boolean operators	8
3.7	If then else	9
3.8	Functions and application	9
3.9	Lists, cons, append and ranges	10
3.10	Tuples	10
3.11	Arrays and grids	10
3.12	Declared types	11
3.13	Patterns	12
3.14	Pattern matching	12
3.15	Comprehensions	13
3.16	The pipe operator	14
4	Type inference	14
5	Borrow checking	15
6	Comparison with Java	16
7	Future work	16

1 Introduction

Nomad is a small, strict, statically typed functional language in the ML family. A program is a list of top-level bindings and expressions. The interpreter reads the whole program, gives every expression a type, and only then runs it. No Nomad program carries a type annotation, because the checker works out every type on its own.

The aim is a language that solves programming challenges quickly. Someone working through a contest problem or an exercise set wants to write the idea down and get an answer. That person should not have to declare types twice, wrap input in a channel object, or spell out a fold that one line of set-builder notation would say. Nomad tries to keep the guarantees a strong type system gives while asking for none of the paperwork.

This report documents the language as the interpreter runs it today. Every example below was run, and the output shown beside it is what Nomad printed. The parser is a library of combinators in the style Hutton and Meijer set out [1].

Borrow checking is the reason the project exists. Section 5 gives the rule it enforces and what that rule buys, and Section 7 lists the work still to come.

1.1 A first program

The program below sorts a list. It uses most of what the language offers: recursive bindings, functions of several arguments, list patterns and comprehensions.

```
let rec quicksort lst =  
  match lst with  
  | [] -> []  
  | x :: xs ->  
    quicksort [y | y <- xs, y <= x] @ [x] @ quicksort [y | y <- xs, y > x]  
  
let result = quicksort [5; 2; 9; 1; 5; 6]
```

Nothing in it names a type, and the checker still rejects it if any part does not fit. The two comprehensions are doing the work that two auxiliary filtering functions would otherwise have to do.

1.2 Running a program

The interpreter takes a file. Source files use the extension `.nd`.

```
$ nomad sort.nd  
<fun> : int list -> int list  
[1; 2; 5; 5; 6; 9]
```

Every statement prints the value it produced, binding forms included, which is why the two lines above correspond to the two bindings in the program. Values print as the source would write them,

with strings in quotes and lists in brackets. A function has no useful printed form, so it shows as `<fun>` followed by its type.

Run `nomad` with no file and it reads statements one at a time instead. The rest of this report uses that prompt to show what an expression produces.

```
$ nomad
>> let xs = [1; 2; 3]
[1; 2; 3]
>> [x * x | x <- xs]
[1; 4; 9]
```

2 Syntax

2.1 Grammar

A program is a sequence of statements. Each statement either binds a name or evaluates an expression. A binding may take parameters, which is shorthand for a function that returns a function.

In the grammar below, text in quotes is typed literally and a bare word is a category defined here, except for `name`, `conname`, `tvar`, `integer` and `string`, which Section 2.3 defines. Parentheses group, `|` separates alternatives, and a trailing `*`, `+` or `?` means zero or more, one or more, and optional.

```
program = statement*

statement = "let" "rec"? name name* "=" expr | expr
           | "type" params name "=" variant ("|" variant)*

params = (tvar | "(" tvar ("," tvar)+ ")")?
variant = conname atom*
atom = tvar | name | "(" type ")"
type = atom name* | "(" type ("," type)+ ")" name+
      | type ("*" type)+ | type "->" type

expr = "if" expr "then" expr "else" expr
      | "fun" name name* "->" expr
      | "let" "rec"? name name* "=" expr "in" expr
      | "match" expr "with" "|" case ("|" case)*
      | expr operator expr | "-" expr | expr expr
      | "(" expr ")" | "(" expr ("," expr)+ ")"
      | "[" (expr ("," expr)*)? "]"
      | "[" expr ("," expr)? ".." expr "]"
      | "[" expr "|" qualifier ("," qualifier)* "]"
      | expr "[" expr "]" | expr "[" expr "]" ":@" expr
      | name | conname | integer | string | "true" | "false" | "()"

case = pattern "->" expr

qualifier = pattern "<-" expr | expr | "let" name name* "=" expr
```

```

pattern = "_" | name | integer | string | "true" | "false"
          | conname apat* | "[" "]" | pattern ":" pattern
          | "(" pattern ("," pattern)+ ")"
apat = pattern with no arguments of its own, or one in parentheses

operator = "||" | "&&" | ">" | "@" | "::"
           | "<" | "<=" | ">" | ">=" | "=" | "<>"
           | "+" | "-" | "*" | "/" | "%"

```

The `expr operator expr` line is deliberately loose. It says which operators exist but not how they group, because that is what Table 1 in the next section is for.

2.2 Operators

Types are written the way the interpreter prints them. `int`, `bool`, `string` and `unit` are the base types, `t list` is a list, `t array` and `t grid` are the two mutable forms, `t1 -> t2` is a function, `(t1 * t2)` is a pair, and `'a` stands for a type that can be anything at all. Types never appear in Nomad source.

Table 1 gives every operator, what it accepts and what it produces.

Level	Operators	Type	Note
1 (loosest)	<code> </code>	<code>bool -> bool -> bool</code>	left, short circuits
2	<code>&&</code>	<code>bool -> bool -> bool</code>	left, short circuits
3	<code>< <= > >=</code>	<code>int -> int -> bool</code>	chained
3	<code>= <></code>	<code>'a -> 'a -> bool</code>	chained
4	<code> ></code>	<code>'a -> ('a -> 'b) -> 'b</code>	left, pipe
5	<code>@</code>	<code>'a list -> 'a list -> 'a list</code>	right, append
6	<code>::</code>	<code>'a -> 'a list -> 'a list</code>	right, cons
7	<code>+ -</code>	<code>int -> int -> int</code>	left
8	<code>* / %</code>	<code>int -> int -> int</code>	left
9	<code>- (prefix)</code>	<code>int -> int</code>	negation
10 (tightest)	application	<code>('a -> 'b) -> 'a -> 'b</code>	left

Table 1: Operators, loosest first.

Both operands are evaluated, left one first, except for `&&` and `||`, which look at the right operand only when the left has not already settled the answer.

The four keyword forms `if`, `fun`, `let` and `match` run as far to the right as they can, which the table cannot show. In `if c then a else b + 1` the `else` branch is `b + 1`, and parentheses cut a form short.

2.3 Layout, comments and lexical rules

A statement ends where the next line begins in the first column. Whitespace inside a statement crosses a newline only when the line that follows is indented, so a continuation line must be indented and a top-level binding must not be. Two semicolons end a statement outright, which is the way to put more than one on a line, and they are optional everywhere else.

```
let a = 1 -- one statement
let b = -- another, continued below
    a + 1
let c = 2 ;; let d = 3 -- two on one line
```

A comment runs from `--` to the end of the line and counts as whitespace. Blank lines and comment lines are looked past when deciding whether the next line is indented, so a comment in the first column does not end the statement above it.

A name starts with a lowercase letter and continues with letters, digits and underscores. A constructor name is the same but starts with a capital, which is what tells the two apart with no declaration in scope. A type variable is a name with a quote in front of it, as in `'a`. The words `let`, `in`, `fun`, `if`, `then`, `else`, `match`, `with` and `type` are reserved.

An integer literal carries no sign, since a leading minus is the prefix operator of level 9. This is the first of the two places spacing matters, and the rule is the one Haskell uses: `1 - 2` and `1 -2` are both subtraction, `1 -2` is 1 followed by a comment, and a negative right operand needs a space, as in `1 - -2`. A pattern keeps the signed literal, since a sign in a pattern cannot be an operator.

A string literal is any run of characters between two double quotes. There are no escape sequences.

3 The language

3.1 Literals

There are four kinds of literal.

```
>> 42
42
>> true
true
>> "hello"
"hello"
```

Integers are machine integers, 63 bits wide on a 64-bit host, and they wrap on overflow. The fourth is the unit value `()`, the only value of type `unit`, which stands for a result that carries no information.

3.2 Variables and bindings

A `let` names a value. At the top level it stays in scope for the rest of the program, and with `in` it stays in scope for one expression. There is no way to declare a name without giving it a value.

```
>> let x = 21
21
>> x * 2
42
>> let y = let a = 2 in a * a
4
```

The bound expression runs once, whether or not the body uses it. A `let rec` puts the name in scope inside its own definition, which is the only way to write a loop, since the language has no loop construct.

```
>> let rec fact n = if n = 0 then 1 else n * fact (n - 1)
<fun> : int -> int
>> fact 5
120
```

Two functions that call each other cannot be written, because neither is in scope in the other's definition.

3.3 Arithmetic

The five arithmetic operators take integers and give integers. Division truncates towards zero and the remainder takes the sign of the left operand.

```
>> -7 / 2
-3
>> -7 % 3
-1
>> 7 % -3
1
```

A zero right operand of `/` or `%` stops the program with **Evaluation Error: Division by zero**. Prefix minus is subtraction from zero.

3.4 Comparison and equality

The four relational operators compare integers. Equality and inequality compare two values of any one type, so both sides must agree but that type is otherwise free.

```
>> [1; 2] = [1; 2]
true
>> (1, true) <> (1, false)
true
```

Equality is structural, so two values of the same type are equal when they are built the same way. Reading two functions cannot decide whether they agree, so comparing them fails at run time with **Evaluation Error: Cannot compare functions**, and a function nested inside a list is caught the same way. Making that a static error would need a notion of equality type, which the language does not have.

3.5 Chained comparison

Several comparisons in a row are sugar for the conjunction of the comparisons between neighbours. The parser does the expansion, so `a < b < c` becomes `a < b && b < c` before anything else looks at it. This gives `0 <= x < 5` the reading it has in mathematics.

```
>> let x = 3
3
>> 0 <= x < 5
true
>> 1 = 1 = 1
true
```

The expansion pairs each operand with both of its neighbours and joins the results with `&&`, grouping to the left. A single comparison is the case that expands to itself. Equality sits on the same precedence level, so `a = b = c` expands to `a = b && b = c` and asks whether all three agree, rather than comparing the boolean `a = b` to `c`.

Each interior operand appears twice in the expansion, so it is evaluated twice. Nothing in Nomad has a side effect, so that costs only time, but a language with input or mutation would have to bind the operands first. Most languages have no such reading. Java rejects `0 <= x < 5` outright, because `0 <= x` is a boolean and nothing compares a boolean to 5, so the bound has to name `x` twice.

3.6 Boolean operators

`&&` and `||` take two booleans and give a boolean. Each one reads its right operand only when the left has not already settled the answer, so `false && e` is `false` and `true || e` is `true` without `e` running at all. A test on the left can therefore guard the expression on the right.

```
>> let n = 0
0
>> n <> 0 && 10 / n > 1
false
>> n = 0 || 10 / n > 1
true
```


Both lines would divide by zero if the right operand ran, and neither does.

3.7 If then else

An `if` picks one of two branches. It is an expression, so it always produces a value and the `else` branch is never optional. Both branches must have the same type, since the checker cannot know which one will run, and only the branch taken is evaluated.

```
>> if 3 > 2 then "yes" else "no"
"yes"
```

3.8 Functions and application

A function takes one argument. Writing several parameters is shorthand: `fun x y -> e` is `fun x -> fun y -> e`, and `let f x y = e` binds `f` to the same thing. Applying such a function to fewer arguments than it expects gives back a function that remembers the ones supplied so far. Parameters carry no annotation, and the checker works out their types from how the body uses them.

```
>> let add x y = x + y
<fun> : int -> int -> int
>> let inc = add 1
<fun> : int -> int
>> inc 10
11
```

A function evaluates to a closure at once and the body waits until the function is applied. The closure keeps the environment the function was written in, which is what makes the language lexically scoped. A free variable in the body means what it meant where the function appears in the source, not what it happens to mean where the function is called, so rebinding `n` below does not reach inside `f`.

```
>> let n = 10
10
>> let f = fun x -> x + n
<fun> : int -> int
>> let n = 0
0
>> f 1
11
```

Application is call by value. The evaluator reduces the function, then the argument, and runs the body in the closure's environment extended with the argument. A recursive closure also puts its own name back in scope before running the body, which is what lets the function call itself without a mutable cell.

3.9 Lists, cons, append and ranges

The list is the only compound data structure the language has. A list literal writes its elements between brackets separated by semicolons, and every element must have the same type. Cons puts one value on the front of a list, append joins two lists, and a range writes a run of consecutive integers with both ends included.

```
>> 1 :: [2; 3]
[1; 2; 3]
>> [1; 2] @ [3] @ [4]
[1; 2; 3; 4]
>> [1..5]
[1; 2; 3; 4; 5]
>> [9, 7..1]
[9; 7; 5; 3; 1]
```

A range counts up by one unless a second element is given, in which case the gap between the two sets the stride and may run downwards. A range whose bound is already past its start is empty, so `[3..1]` is `[]`, and a stride of zero stops the program. The empty list leaves its element type free, so it can serve as a list of anything. Cons and append both build a new list and leave their arguments alone. Append groups to the right, so `a @ b @ c` reads as `a @ (b @ c)` and each append walks only its own left operand rather than the result accumulated so far.

3.10 Tuples

A tuple groups a fixed number of values of any types at all. The parentheses are always required, which is what keeps this comma from clashing with the one separating comprehension qualifiers, and a single parenthesised expression is that expression rather than a tuple of one.

```
>> (1, "a", true)
(1, "a", true)
```

Width is part of the type, so a pattern of the wrong width does not check.

3.11 Arrays and grids

An array holds a fixed number of values of one type and reaches any of them in constant time. It is the one thing in the language a program updates rather than rebuilds. Reading is `a[i]` and writing is `a[i] := v`, which gives back the array.

```
>> let a = array 3 0
[|0; 0; 0|]
>> let b = a[1] := 9
[|0; 9; 0|]
>> b[1] + size b
12
```

array `n v` builds one with every slot `v`, `size` measures it, `from_list` turns a list into one and `copy` duplicates it. An index outside it stops the program. The bracket has to follow with no space, the second place spacing matters, so `f [1; 2]` is an application and `a[1]` an index.

A grid is two-dimensional and held in one block. `grid r c v` builds one, `rows` and `cols` measure it, and `g[i][j]` reads and writes a cell.

```
>> let g = grid 2 3 0
      [|10; 0; 0|]; [|0; 0; 0|]|
>> let h = g[1][2] := 7
      [|10; 0; 0|]; [|0; 0; 7|]|
```

An array may not be stored inside anything else, so there are no arrays of arrays and none inside a list or a tuple. A grid is what covers the two-dimensional case, and it costs nothing to check because its cells sit in one block rather than in a row of separate arrays. Section 5 says why the restriction is there.

3.12 Declared types

A declaration names a type and lists the ways to build a value of it. Each alternative is a constructor, written with a capital, followed by the types of the arguments it takes.

```
>> type shape = Circle int | Rect int int
>> Circle 3
Circle 3
>> Rect 3 4
Rect 3 4
```

A constructor takes its arguments one at a time, like any other function, so it can be given fewer than it needs and passed around.

```
>> Circle
<fun> : int -> shape
```

A declaration may name the type it is declaring, which is what gives the language recursive structures. It may also take type variables, written with a leading quote. One goes on its own and several go in parentheses separated by commas, which is how they are supplied where the type is used too.

```
>> type tree = Leaf | Node tree int tree
>> Node (Node Leaf 1 Leaf) 2 Leaf
Node (Node Leaf 1 Leaf) 2 Leaf
>> type 'a option = None | Some 'a
>> Some 1
Some 1
>> type ('a, 'b) either = Left 'a | Right 'b
```

The arguments of a constructor are written as single types, so an applied type among them needs parentheses, as the trie in `examples/trie.nd` does with `Node bool ((int * trie) list)`. A

declaration produces no value and prints nothing, and `'a option` above is an ordinary declaration rather than anything the language knows about.

3.13 Patterns

A pattern describes the shape of a value and names the parts worth keeping. There are nine forms.

Pattern	Matches	Binds
<code>-</code>	anything	nothing
<code>x</code>	anything	<code>x</code>
<code>42</code>	that integer	nothing
<code>true</code>	that boolean	nothing
<code>"ok"</code>	that string	nothing
<code>[]</code>	the empty list	nothing
<code>p1 :: p2</code>	a list with at least one element	what <code>p1</code> and <code>p2</code> bind
<code>(p1, ..., pn)</code>	a tuple of that width	what each part binds
<code>Con p1 ... pn</code>	a value built by that constructor	what each part binds

Table 2: Pattern forms.

A name may appear only once in a pattern, since there would be no reason to prefer one of the two values it caught. What a pattern binds is visible in the body it guards and nowhere else, and a pattern variable hides an outer variable of the same name. A constructor pattern must supply exactly as many arguments as the constructor takes, and each of those has to be a pattern with no arguments of its own or one in parentheses, which is what keeps `Node Leaf 1 Leaf` from reading as one deep pattern.

3.14 Pattern matching

A `match` tries its cases from top to bottom and runs the first one that fits. One case per line is the usual layout, and a bar may stand before the first case as well as between them, which lets every case line up under the same column. Every case must produce the same type and every pattern must match the type of the value being matched.

Taking a list apart is the common use. The first case handles the empty list and the second names the head and the tail.

```
let rec length l =
  match l with
  | [] -> 0
  | _ :: t -> 1 + length t
```

With the constant patterns a `match` also serves where other languages use a switch. A tuple pattern takes a tuple apart in one case, and a declared type takes one case per constructor.

```

let describe n =
  match n % 3 with
  | 0 -> "none left over"
  | _ -> "something left over"

let rec total t =
  match t with
  | Leaf -> 0
  | Node l x r -> total l + x + total r

```

The cases must cover every value the matched expression could take. A match that leaves one out is rejected before the program runs, because Nomad has no exceptions and no way to report a failed match at run time.

```

>> let head l = match l with x :: _ -> x
Type Error: This match is not exhaustive

```

The checker decides this with Maranget’s usefulness algorithm [7]: the patterns are exhaustive exactly when adding a further wildcard case would be useless. The type says which sets of constructors are complete, so both booleans suffice for a boolean, `[]` together with `::` suffice for a list, one tuple pattern of the right width suffices for a tuple, and a declared type is covered by all of its constructors, while integers and strings are too many to cover without a wildcard.

3.15 Comprehensions

A comprehension writes a list by saying what its elements look like rather than by folding over another list. This is set-builder notation, so the bar reads as *such that*.

```

>> [x * x | x <- [1..10]]
[1; 4; 9; 16; 25; 36; 49; 64; 81; 100]
>> [x | x <- [-2; 3; -4; 5], x > 0]
[3; 5]
>> [y | x <- [1; 2; 3], let y = x * 10, y > 10]
[20; 30]
>> [x * 10 + y | x <- [1; 2], y <- [3; 4]]
[13; 14; 23; 24]
>> [h | h :: t <- [[1; 2]; []; [3]]]
[1; 3]

```

There are three kinds of qualifier: a generator `p <- e` draws values from a list, a guard is any boolean expression, and a `let` names a value. They run left to right and nest, so the leftmost generator is the outer loop and a later qualifier sees every name an earlier one bound. A guard skips the qualifiers after it for the value in hand. A generator pattern may be refutable, and a value it does not match is skipped rather than being an error, which is the one place the exhaustiveness condition deliberately does not apply. That is what the last line above relies on to take the heads of the non-empty lists.

The language is strict, so the whole list is built before anything else happens. Nomad does not desugar into folds the way Wadler’s translation does [9], because there is no standard library to desugar into and a construct of its own gives errors that point at what the programmer wrote.

A comprehension whose body is an update fills an array instead of collecting a list, and its value is the array. The qualifiers behave exactly as above, so a guard that fails leaves the array as it was.

```
>> let a = array 4 0
      [0; 0; 0; 0]
>> [a[i] := i * i | i <- [1..3]]
      [0; 1; 4; 9]
```

Nothing new is written down for this. Collecting a list of the array would have type `'a array list`, which Section 3.11 rules out, so the form had no other meaning to take away. Section 6 fills a knapsack table this way, `examples/bellman_ford.nd` relaxes its edges, and `examples/edit.nd` fills a grid with its base cases and its recurrence in one comprehension.

3.16 The pipe operator

The pipe writes a call with the argument on the left, so a chain of steps reads in the order the data flows. Instead of `h (g (f x))`, write `x |> f |> g |> h`. It is sugar, rewritten by the parser into `e2 e1`, and it groups to the left.

```
>> let double x = x * 2
<fun> : int -> int
>> [1; 2; 3] |> length |> double
      6
```

4 Type inference

A Nomad program carries no annotations, so the checker finds every type by itself. It uses the standard method: give the unknowns names, collect the constraints the program forces on those names, and solve them by unification. Pierce gives the full treatment [2], and the Cornell textbook works the same algorithm through in OCaml [3].

A type variable stands for a type nobody has pinned down. The checker makes a fresh one whenever it meets an unknown, such as a function parameter or the element type of an empty list, and unification then takes two types and finds the most general way to make them equal, or fails. The algorithm is Robinson’s [4]. A variable must not be made equal to a type that contains it, since that would describe a type with no finite form. The occurs check rejects it.

```
>> fun x -> x x
Type Error: In x x: Recursive types not supported
```

A binding does not get one type. It gets a *scheme*, which is a type together with the variables in it that were left free, and every use of the name takes a fresh copy of those. This is full Hindley-Milner inference [5, 6], and it is what lets one definition serve several types.

```
>> let id x = x
<fun> : 'a -> 'a
>> (id 1, id "a")
(1, "a")
```

Without it no standard library could exist, since one `map` could not serve lists of integers and lists of strings in the same program. A top-level binding and a comprehension `let` generalise the same way.

Function parameters and pattern variables do not generalise. Their types are fixed by the caller, not by the binding, so a parameter used at two types is rejected, as it should be.

```
>> let bad = fun f -> if f true then f 1 else 0
Type Error: In f 1: Type mismatch: bool and int
```

Nomad needs no value restriction yet. An array is the one thing that mutates, and Section 5 stops any name from observing the change, so no generalised type can be wrong about it. A reference the programmer could alias would need the restriction.

5 Borrow checking

An array is the one mutable thing in the language, and one rule keeps it behaving like a value. Reading with `a[i]` borrows the array and leaves it usable, so it can be read as often as it likes. Writing with `a[i] := v` gives it away, and so does passing it to a function that writes to it. A name that has been given away may not be used again.

```
>> let a = array 3 0
[|0; 0; 0|]
>> let b = a[0] := 1
[|1; 0; 0|]
>> a[0]
Borrow Error: a was already given to an update
```

copy is how a program keeps a version it still needs, which puts the cost of keeping both in the source where the reader can see it.

Because nothing can observe an array after it has been given away, an update overwrites in place rather than copying. That is the whole prize: a loop of n updates is linear rather than quadratic, reading is constant time, and no two names ever stand for one array.

Which arguments a function gives away is worked out from its body rather than written down, so no type changes. `let bump x = x[0] := 1` still reports `int array -> int array`, and the checker separately knows that a caller loses its array to it.

Three restrictions keep the rule sound. A function that writes to an argument has to be called with all of its arguments rather than passed around as a value, since nothing then knows what it does. A function written inline may not write to an array at all, because it may run more than once. And an array may not be stored inside another structure, which is what closes aliasing: two names for one array would let a write through one be read through the other, and the checker follows names rather than paths. The comprehension of Section 3.15 is the deliberate exception to the second restriction, because it is what produces the array.

The discipline is affine rather than linear, since an array may be dropped without being written to, and it is the same idea Wadler puts to work for in-place update [8]. Rust spends it on freeing memory without a collector. Nomad has a collector already, so it spends it on keeping value semantics at array speed.

6 Comparison with Java

Nomad exists to state the answer to a programming challenge in as few lines as it takes to think of it, and Java is the language most students reach for. This compares only what Nomad already has, since input and a standard library are not written yet, so it is not a scoreboard. Both programs below were run.

A comprehension states a condition and says nothing about how to search, so `examples/triples.nd` finds every Pythagorean triple in two lines where Java writes three nested loops, picks a container and wraps it all in a class with a `main`. The comparison below is the one where the difference is not just ceremony.

The 0/1 knapsack. Nomad fills one array with a comprehension whose body is an update, and the descending range lets a single row stand in for the whole table. Java writes the same two loops out and reaches into the item with an index, where the generator pattern names the two fields.

Nomad

```
let knapsack cap items =  
  let t = array (cap + 1) 0 in  
  [t[w] := max t[w] (t[w - wt] + v1)  
   | (wt, v1) <- items,  
   w <- [cap, cap - 1..wt]][cap]
```

Java

```
static int knapsack(int cap, int[][] items) {  
  int[] t = new int[cap + 1];  
  for (int[] it : items)  
    for (int w = cap; w >= it[0]; w--)  
      t[w] = Math.max(  
        t[w], t[w - it[0]] + it[1]);  
  return t[cap];  
}
```

Both give 7 for a capacity of 5 and the items (2,3) (3,4) (4,5) (5,6). The update is in place in both, but only one of them is checked: Java would let a later line read `t` after handing it to something that writes to it, and Section 5 makes that a compile error.

7 Future work

Here is a non-exhaustive list of features I would like to implement beyond this course.

- **A conditional expression.** `x > 0 ? 1 : -1`, sugar for `if`.
- **Parallel generators.** `[x + y | x <- xs | y <- ys]`, walking two lists in step.
- **Comprehensions as functions.** `[x * 2 | x <- _]`, with the hole marked rather than guessed.
- **Collectors.** `sum[x * x | x <- [1..n]]`, so the notation can fold as well as build, the way SRFI 42 does [10].
- **A standard library.** Every program currently writes its own `concat` and `length`.
- **Arrays inside other structures.** Today an array may not sit in a tuple, so no function can hand back a table together with a result. Memoised recursion and a one-pass union-find both want that, and it needs the checker to follow paths rather than names.
- **Native grid operations.** Rows, columns and transposition on the grids of Section 3.11, along the lines of [11].
- **Input and output.** Reading input should take one line, the way `cin >> x >> y` does in C++.

References

- [1] Graham Hutton and Erik Meijer. *Monadic Parser Combinators*. Technical Report NOTTCS-TR-96-4, Department of Computer Science, University of Nottingham, 1996.
<https://people.cs.nott.ac.uk/pszgmh/monparsing.pdf>
- [2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [3] Michael R. Clarkson et al. *OCaml Programming: Correct + Efficient + Beautiful*. Cornell University, CS 3110. <https://cs3110.github.io/textbook/cover.html>
- [4] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.
- [5] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.
- [6] Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM Symposium on Principles of Programming Languages*, pages 207–212, 1982.
- [7] Luc Maranget. Warnings for pattern matching. *Journal of Functional Programming*, 17(3):387–421, 2007.
- [8] Philip Wadler. Linear types can change the world! In *Programming Concepts and Methods*, North Holland, 1990.
- [9] Philip Wadler. List comprehensions. Chapter 7 of Simon L. Peyton Jones, *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.
- [10] Sebastian Egner. *SRFI 42: Eager Comprehensions*. Scheme Requests for Implementation, 2003.
<https://srfi.schemers.org/srfi-42/srfi-42.html>
- [11] Jacob Errington. *Grids without indices*. 2025.
<https://jerrington.me/posts/2025-12-23-grids-without-indices.html>